

Why an agentic harness is a comonad

Predicting what an agent will do, and proving compaction didn't change it

An agentic harness is the program that lets a language model actually read files, run commands, and make edits. Built the usual way it is an imperative loop; built as a coalgebra unfolded into a comonad it becomes something you can interrogate — you can predict what the agent will do before it does it, and state and check that compacting its history did not change its behaviour.

The Haskell names below (`step`, `unfold`, `governed`, `extend`, `assess`, `admit`, `outerLoop`, `compact`, `respectsBehaviour`) refer to the accompanying library. Snippets shown in another language are from the `pi` agent harness (TypeScript), simplified for illustration.

What an agentic harness has to do

An agentic harness turns something that only emits text into something that reads files, runs commands, and makes commits. Strip it to the job and it has five obligations — offer the model a set of tools and a view of the conversation; carry out (or refuse) what it asks for; feed the results back and go round again; keep the conversation inside the model's context window as it grows; and stop, eventually, with an outcome.

Everything else — streaming, retries, a UI — is comfort around those five. The interesting question is not how to write the loop (it is a loop) but what you are allowed to *say* about it once written: can you predict what it will do? can you show that squeezing the history did not change its behaviour? Most harnesses answer “no” to both and ship anyway. One shape — a coalgebra unfolded into a comonad — turns those “no”s into “yes”.

How the canonical harness does it

The `pi` harness is a clean, minimal example of how this is normally done, and most others (`tau` among them) are the same shape. The core is an imperative loop:

```
while (true) {                                     // stop conditions elided
  const message = await streamAssistantResponse(currentContext, ...);
  const toolCalls = message.content.filter((c) => c.type === "toolCall");
  if (toolCalls.length > 0) {
    const results = await executeToolCalls(currentContext, message, ...);
    for (const result of results) currentContext.messages.push(result);
  }
}
```

Ask the model, run the tools it asked for, push the results onto a growing messages array, repeat. Much of it agrees with the account below. Tools are plain data dispatched by name, and an unknown or malformed call is repaired into an error handed back to the model rather than crashed on:

```
const tool = currentContext.tools?.find((t) => t.name === toolCall.name);
if (!tool) {
  return { result: createErrorToolResult(`Tool ${toolCall.name} not found`) };
}
```

That is exactly the move `admit` makes in the coalgebra — an unafforded call becomes a synthetic error observation, not an exception. Compaction is there too, fired on a token threshold:

```
function shouldCompact(contextTokens, contextWindow, settings) {
  return contextTokens > contextWindow - settings.reserveTokens;
}
```

So the coalgebra is not exotic: the practical machinery matches a working agent.

What differs is what you can *say* about the loop, and it follows from one fact — the control flow only exists while it runs. The loop *is* the interpreter, and the model and tool calls happen inside it, so there is no value standing for “what the agent will do” that you could inspect without executing. Three things fall out. You cannot look before you leap: there is no way to ask what the agent will do next short of letting it — a “plan mode” is a tool-gating convention, not a forecast. Compaction is best-effort: it fires on a threshold and is judged by whether the session still works afterwards, never against a statement of what it must preserve. And behaviour is checked by anecdote — canned fake providers and example transcripts — because there is no closed set of actions or shared interpreter to state a property over.

What turns those three “no”s into “yes” is treating the control flow as a value instead of a running process.

Two directions on one tree

Stop writing the loop and start writing the thing the loop walks. Give the harness a pure coalgebra `step :: S -> HarnessF S` — from a state, the single next action — and `unfold` it into `Cofree HarnessF Ctx`, the lazily-grown tree of every reachable future. Nothing runs; you have built a value.

That value has two directions on it, and they map onto an idea from John Boyd — the US Air Force strategist best known for the OODA loop (observe, orient, decide, act) — set out in his 1976 essay [Destruction and Creation](#). *Creation* is the `unfold`: synthesise the whole branching future from a seed coalgebra, a model of what could happen. *Destruction* is the `fold`: take that tree apart to see what each state’s future actually holds. Boyd’s argument is that staying adaptive means doing both, forever — you cannot analyse without first pulling a whole into parts, and you cannot act without assembling parts into a new whole. A harness that can only run forward — create, never destruct its own futures — cannot govern itself. The canonical loop is all creation: it acts, and never inspects.

The comonad is the destruction direction made mechanical. You cannot fold an infinitely-branching tree directly, so you fold it under a *hypothesis*: a pure stand-in for the oracle and world that collapses each branch to the one path it would take. Under a hypothesis the tree is a stream, `extract` reads the annotation at the root, and `extend` rewrites every node with a function of its *whole* sub-future. Those are the comonad operations, and they are the vocabulary of analysis the canonical loop has no words for.

A deliberately smaller coalgebra than the real `step` shows the creation seed: a fixed position whose successor is a pure function of the reply.

```
-- The nondeterminism is entirely in the Either Refusal Response argument;
-- the position (Render) is fixed, only the reply varies.
exampleStep :: S -> HarnessF S
exampleStep s = Render (request s) reply
  where
    reply (Left _) = s { budget = 0 }
    reply (Right r) = s { transcript = Assistant r : transcript s }
```

What destruction buys: looking before you leap

Concretely: write `assess`, a function that scores one node from its own future — how far ahead it halts, whether it terminates at all, which irreversible tools it touches, how many compactions it forces. Then `governed = extend (assess h n)` lifts that one-node score to *every* reachable node, and `extract`

(governed h n w) is the score at the current node: a pre-action gate you can consult before letting the agent move.

That is the first “no” turned to “yes” — you can look before you leap. And because `run` (the real thing, in IO) and `probe` (a pure, bounded dry-run) walk the same tree through one interpreter, the future the gate shows agrees with the run that follows *by construction*, not by hope. (The comonad operation `extend` is what `governed` uses; the score at a single node is `assess`, which the next section reuses without lifting it over the tree.)

Creation again, on a computed trigger

Boyd’s loop does not stop at analysis — the point of destruction is to feed a new creation. When the forecast says a coalgebra will not terminate, the harness swaps it for a weaker one and unfolds again. `evolve` builds a chosen coalgebra from a seed; `outerLoop` tries successively weaker coalgebras and runs the first whose *forecast* — an `assess` of an unfolded-but-unrun tree — says it halts. This is destruction-creation with a computed trigger rather than a guessed one, and the comonad supplies the destruction half that makes the trigger computable. (`outerLoop` reads `assess` on one tree rather than lifting `extend` over it; the rhythm is Boyd’s either way.)

Why the shape allows it

None of this works if analysis is effectful, and that is bought by one decision: put the nondeterminism in the *directions*, not the *positions*. A `Render` position is a fixed `Request`; what the harness does not control is the reply, so the successor is a *function* of an `Either Refusal Response` it has not seen. Because a branch is a function of an unobserved input rather than an effect that must be run, `duplicate` and `extend` on the tree are pure and lazy — you can walk futures without touching the world. The standard objection, that an agent must run tools to enumerate its futures, dissolves: the tree branches on unobserved *inputs*, not unperformed *effects*.

Driving that fragment one step with a supplied (pure) reply makes the point without any effect. The `case` is exhaustive over all three constructors on purpose — the action set is closed, and a wildcard would hide the day a fourth is added.

```
exampleRun :: S -> Either Refusal Response -> S
exampleRun s r = case exampleStep s of
  Render _ k -> k r
  Perform _ k -> k (Obs "")    -- unreachable for this fragment; kept total
  Halt _      -> s
```

Compaction as bisimulation

The second “no” was compaction. Collapsing the transcript to stay inside the window should be a *coalgebra homomorphism*: `step (compact s) == fmap compact (step s)`. Read that as: `s` and `compact s` are *bisimilar* — they produce the same behaviour under the coalgebra, even though `compact s` shows the model a shorter prompt. That is the precise content of the slogan *compaction is correct exactly when it is invisible in what the agent does, not in what it sees*. You cannot check the homomorphism directly (`HarnessF S` holds functions), so you check its behavioural shadow — and the shadow that matters most is the multiset of irreversible actions, since a compaction that manufactures or drops a write has changed behaviour whatever it did to the prompt.

```
-- A compaction that changes this multiset has changed behaviour, which is the
-- divergence respectsBehaviour is built to catch.
irreversible :: [Turn] -> [String]
irreversible = concatMap turnWrites
  where
    turnWrites (User rs) = [tool c | (c, _) <- rs, tool c `elem` writers]
```

```
turnWrites _ = []  
writers = ["write", "commit"]
```

The check is only lawful because the action set is *closed*. There is exactly one interpretation of `HarnessF`, shared by execution and analysis, so “same behaviour” is a property you can state and test — not a coincidence hoped for across two hand-written loops (the failure mode the canonical harness lives with, where “plan mode” and the real run are the same loop under different prompts). `irreversible` is that one shadow as a tiny fold, the same notion `respectsBehaviour` checks at scale.

What it does not buy

Honesty about the trade. The comonad supplies the plumbing to attach a forecast cheaply; it does not supply a model of the oracle, and the forecast is only as good as the hypothesis — against a crude one it gets the shape of the run right and the content wrong. The correctness law is real but state-dependent: checked at a single state it can report “behaviour preserved?” true by luck, when the compacted and uncompactd runs happen to fall the same side of the hypothesis’s threshold; the honest measure is a violation *rate* over states reached by driving the harness through its own transitions, not a boolean. And the comonad structure is genuinely load-bearing in exactly one place, **governed** — the rest builds the tree with `unfold` and walks its shape.

The claim was never that the comonad does the work. It is that framing the harness this way lets you *say*, and check, the two things the canonical loop cannot: what the agent will do, and that compaction did not change it. The canonical harness is the better product; this is the better account of what an agent run *is*.